Sign Language Translation to English

Rishika Cherivirala, Dimitrios Nerantzinis, Ying Lin Zhao
Github: https://github.com/yzhao2433/sign_lang_recognition

Introduction

Every day, millions of people who use American Sign Language (ASL) encounter barriers when communicating with those who do not understand it, turning simple interactions into challenges. Advances in image recognition offer a powerful opportunity to bridge this gap. Using data from the Sign Language Modified National Institute of Standards and Technology (MNIST) database, we aim to train a neural network capable of accurately classifying images of ASL hand gestures into corresponding English characters. By translating visual language into text in real time, this project has the potential to improve accessibility, foster more inclusive communication, and support technologies that increase accessibility and empower individuals in education, workplaces, and everyday life.

Data Exploration

The original Sign Language MNIST dataset maps labels (0 - 25) to alphabet letters A - Z, excluding J and Z due to their dynamic gestures. This dataset contains 27,455 training samples and 7,172 test samples, each represented as a 28×28 grayscale image flattened into 784 features. The dataset was derived from a set of 1,704 colored hand gestures images from multiple users and backgrounds. To expand the dataset, an image processing pipeline that cropped images to the hand region, converted them to grayscale, resized them, and applied various augmentations such as contrast adjustments and rotations, generated over 50 variations per image.

In preparation for modeling, the training data was split into the train and validation dataset to create a holdoutset that can be used for hyperparameter tuning. Since the validation set influenced the model tuning process, the separate test set was used to provide a final, unbiased evaluation of the fully tuned model's generalization performance on entirely unseen data. All three data sets were normalized using the training set’s pixel mean and standard deviation to avoid data leakage. Normalization places features on a common scale and ensures that most neurons will have non-zero gradients, which helps with effective parameter updates and convergence speed.

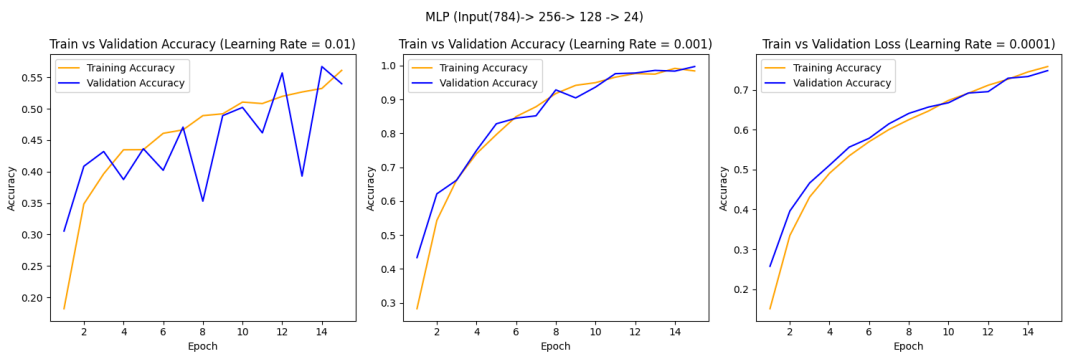
Multilayer Perception (MLP) Model

A Multi-Layer Perceptron (MLP) is a class of fully connected, feedforward artificial neural networks that processes a flattened 1-D input vector through one or more fully connected hidden

layers. While fully connected layers inherently lack spatial awareness, MLPs are highly effective when applied to small and simple images like the images in the dataset we are using. Although the full connected property of MLPs lead to higher number of parameters compared to models such as the convolutional neural network (CNN), the Sign Language image data consists of low-resolution grayscale images, so flattening the 2-D images into 1-D vectors would not lead to a concerning number of parameters. Furthermore, because the hand gestures are relatively centered and consistently framed, the MLP can efficiently map raw pixel intensities directly to class labels. Due to these characteristics, the MLP serves as a benchmark for establishing initial model performance before exploring more complex architectures.

Since the letters J and Z are not included in the model, this multiclass classification problem has 24 distinct classes. To begin, a 3-layer MLP model with $Input(784) \rightarrow 256 \rightarrow 128 \rightarrow 24)$ was trained using ReLU as the activation function with 15 epochs and an Adam (Adaptive Moment Estimation) optimizer with learning rate of 0.01.

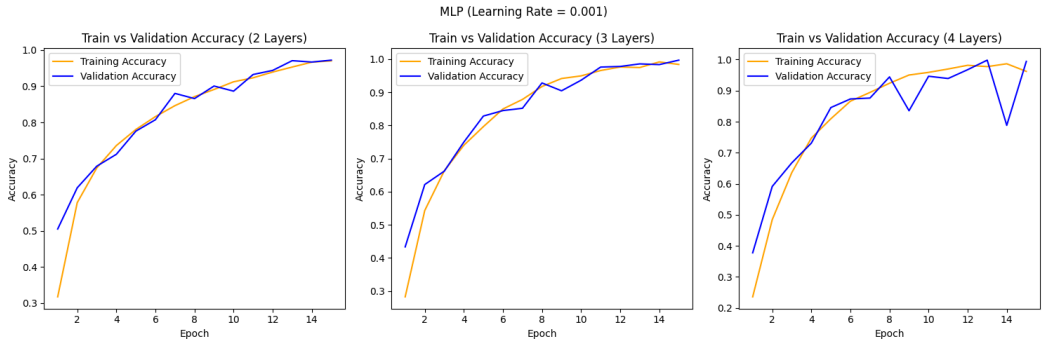
Tuning the Learning Parameter



While the loss decreases overall for both the training and validation sets throughout the epochs, the fluctuations in the train and validation metrics suggest a slight instability in the epochs, likely due to the high learning rate. Although Adam adaptively scales learning rates for each parameter, the initial global learning rate still impacts the overall step size, thus a high learning rate can lead to unstable updates during training. To address this, the learning rate was reduced to 0.0001. As expected, reducing the learning rate from 0.01 to 0.0001 significantly improves the train and validation accuracy, as it approaches 0.75 for both at epoch 15, and leads to smoother accuracy curves. However, the accuracy of 0.75 at epoch 15 indicates a sluggish rate of improvement, likely attributable to the relatively low learning rate. Tuning the learning rate to 0.001 shows to offer a better balance between rate of convergence and stability between the epochs.

Tuning the Depth of the Model

In addition to tuning the learning rate, the model depth can be adjusted, specifically, an additional layer was added to the MLP architecture with the expectation that it would increase the accuracy.

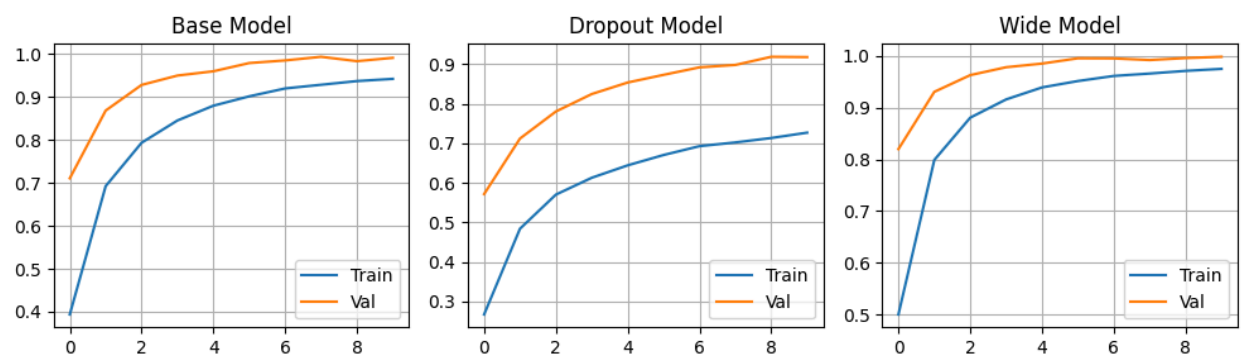


Increasing model depth from 2 to 3 layers improves both training and validation accuracy, with faster convergence and near-perfect performance. However, the 4-layer model shows greater fluctuations in validation accuracy, suggesting increased instability and potential overfitting despite similar peak performance. Overall, a 3-layer architecture provides the best balance between accuracy and stability. Despite good performance on the training and validation sets, the test accuracy is approximately 0.5417, which indicates that the model does not generalize well to completely unseen data, and a more complex model that can capture spatial relationships should be used.

Convolutional Neural Network (CNN) Model

Unlike traditional fully connected networks, Convolutional Neural Networks (CNNs) utilize the spatial structure of images by learning local patterns and hierarchical features through convolutional operations. Because of this, we applied CNN based models to the Sign Language MNIST dataset, where recognizing subtle differences in hand gestures requires capturing detailed spatial patterns. To evaluate the effectiveness of CNNs, we began with a baseline architecture and explored a series of modifications, including regularization techniques, changes in activation functions, and variations in model capacity.

Adjusting Number of Parameters at Each Layer



We began with a baseline CNN that consisted of 2 convolutional blocks followed by fully connected layers. The model processed $28\ x\ 28$ grayscale images through the following pipeline: $Conv(1 \rightarrow 8,\ 3\ x\ 3) \rightarrow Tanh \rightarrow MaxPool(2\ x\ 2) \rightarrow Conv(8 \rightarrow 16,\ 3\ x\ 3) \rightarrow Tanh \rightarrow$

$MaxPool(2\ x\ 2) \rightarrow Flatten \rightarrow FC(32) \rightarrow Tanh \rightarrow FC(25)$. This model achieved a high validation accuracy of 0.996 and served as a good benchmark for future models. However, the training accuracy continued to increase across epochs while the validation accuracy started to plateau, which indicated mild overfitting.

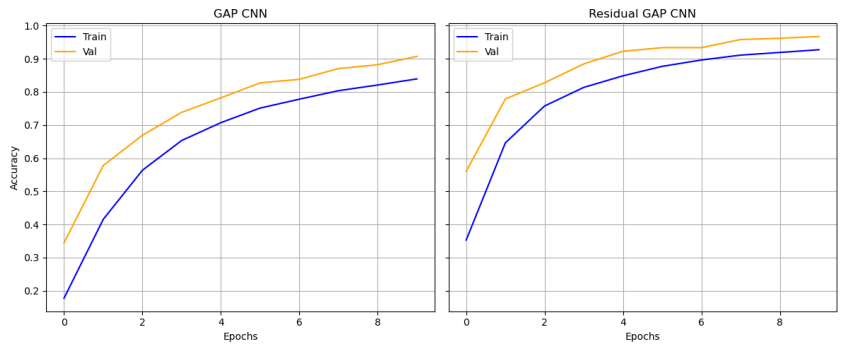
To address the slight overfitting observed in the base model, we introduced dropout layers after each convolutional block. The updated pipeline became: $Conv(1 \rightarrow 8,\ 3\ x\ 3) \rightarrow Tanh \rightarrow MaxPool(2\ x\ 2) \rightarrow Dropout(0.4) \rightarrow Conv(8 \rightarrow 16,\ 3\ x\ 3) \rightarrow Tanh \rightarrow MaxPool(2\ x\ 2) \rightarrow Dropout(0.4) \rightarrow Flatten \rightarrow FC(32) \rightarrow Tanh \rightarrow FC(25)$. Dropout randomly deactivates a subset of activations during training, which encourages the network to learn more robust features and become dependent on feature representations. This model achieved a validation accuracy of 0.936, which is a decrease compared to the baseline model. This suggests that the model began to underfit the data, likely because the dropout rate was too high relative to the model's size. As a result, important spatial features may not have been effectively learned.

Since the baseline model appeared to lack sufficient capacity to fully capture the complexity of the data, we increased the number of filters in the convolutional layers, which widened the network. The new pipeline became: $Conv(1 \rightarrow 32,\ 3\ x\ 3) \rightarrow Tanh \rightarrow MaxPool(2\ x\ 2) \rightarrow Conv(32 \rightarrow 16,\ 3\ x\ 3) \rightarrow Tanh \rightarrow MaxPool(2\ x\ 2) \rightarrow Flatten \rightarrow FC(54) \rightarrow Tanh \rightarrow FC(25)$ By increasing the number of filters, the model is able to learn a more diverse set of features at each layer. This is important especially for sign language classification because subtle differences in hand shape must be captured. Because of this, this model achieved the highest validation accuracy so far with an accuracy of 0.9995 and reached a faster convergence during training compared to the previous two models. However, at the same time, a wider network led to an increase in the total number of parameters to 56809 which is almost double the amount of parameters that the base model utilized and a large number of parameters can lead to poor generalization of unseen data and increased memory usage.

Global Average Pooling (GAP) Model

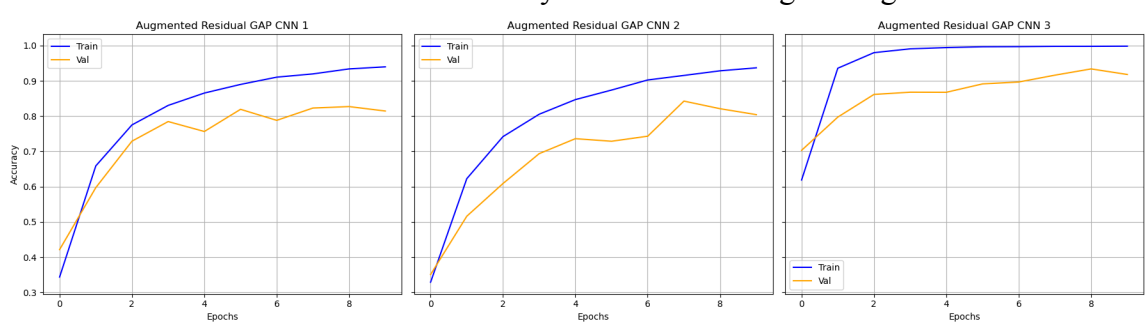
To further improve the model's efficiency and reduce the number of parameters, we first experimented with replacing the flattening step with a Global Average Pooling (GAP) layer. Instead of converting the entire feature map into a long vector, GAP computes the average of each feature map, reducing dimensionality and decreasing the number of parameters in the fully connected layers. However, this modification resulted in a noticeable drop in validation accuracy, suggesting that the model lost some important spatial information during the dimensionality reduction. Because of this, we introduced a residual connection within the convolutional layers to maintain parameter efficiency. Residual connections allow the model to preserve low-level features by adding the input of a layer directly to its output, which improves gradient flow and enables the network to learn more stable feature representations without significantly increasing the number of parameters. The updated pipeline became: $Conv(1 \rightarrow 8,\ 3\ x\ 3) \rightarrow$

$BatchNorm \rightarrow Tanh \rightarrow Conv(8 \rightarrow 8, 3\,x\,3) \rightarrow BatchNorm \rightarrow Tanh + Residual \rightarrow MaxPool(2\,x\,2) \rightarrow GlobalAvgPool + GlobalMaxPool \rightarrow Concatenate \rightarrow Flatten \rightarrow FC(32) \rightarrow Tanh \rightarrow FC(25)$. By combining residual connections with both Global Average Pooling (GAP) and Global Max Pooling (GMP), the model is able to retain important spatial features while still reducing the number of parameters compared to earlier architectures. This design helps mitigate overfitting while capturing both general feature patterns and strong localized activations in the data. As a result, the Residual GAP model achieved strong validation accuracy of 0.9670 while being more parameter efficient than the wider CNN models.

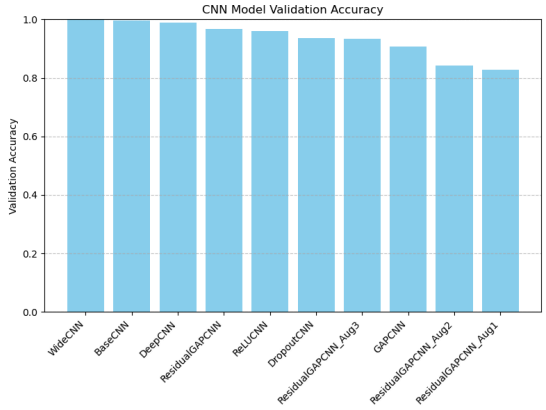


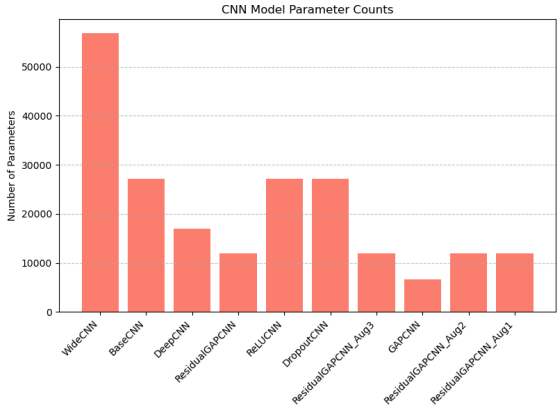
Data Augmentation

To further improve generalization, we applied data augmentation techniques during training using the Residual GAP model, which introduces variability and allows the model to learn more robust and invariant features. We initially applied relatively strong transformations, including random rotations of up to 10 degrees and translations of up to 10% of the image size. However, this resulted in a significant decrease in validation and test accuracy. This is likely because sign language gestures depend on subtle differences in finger positioning, and these aggressive transformations distorted important spatial features. We then refined the augmentation strategy by reducing the intensity of the transformations to smaller rotations (± 5 degrees) and translations. This improved performance, as the model was able to benefit from additional variability without excessively altering the structure of the hand gestures. Finally, we implemented a milder augmentation strategy using very small transformations (± 3 degrees rotation and 5% translation) and combined the augmented dataset with the original training data. By training on both original and lightly augmented images, the model was able to better capture the true data distribution while also becoming more robust to small variations. This final approach achieved the best validation accuracy of 0.9337 among the augmentation models.



The figures below show the validation accuracy and parameter counts for all CNN models. Across the different architectures we tried, there were clear trade-offs that emerged between the model complexity, parameter efficiency, and generalization. While the Residual GAP CNNs offered a parameter-efficient solution that captured important spatial features and benefited from mild data augmentation, the WideCNN achieved the highest validation accuracy (0.9995). This model also generalized well to unseen data, with a test accuracy of approximately 0.9787, confirming that its performance extended beyond the training set.



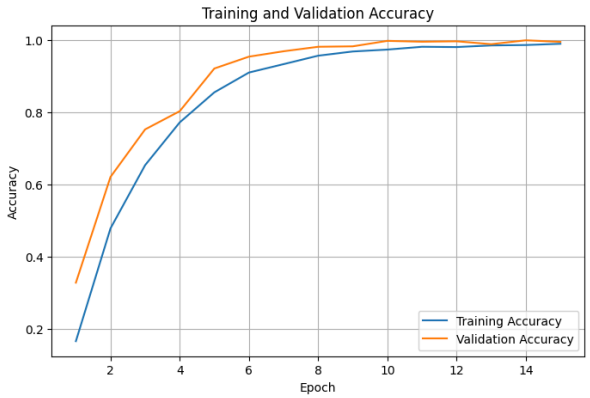


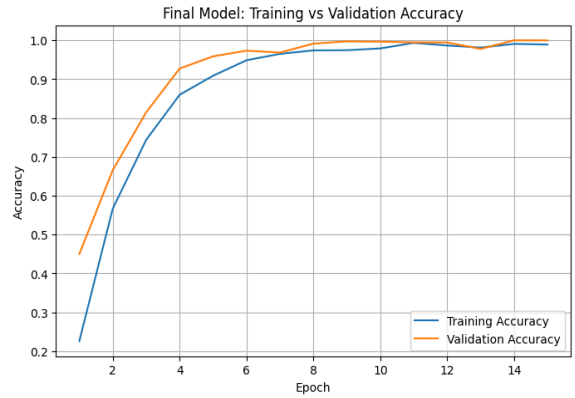
Long Short Term Memory (LSTM)

Although the Sign Language MNIST dataset consists of static images, an LSTM model was implemented to investigate if a sequence-based neural network could effectively learn the spatial structure of hand gestures. LSTMs are designed to capture dependencies across ordered inputs, and by restructuring each 28×28 image into a sequence of 28 rows, the model processes the image one row at a time. In this case, each row is treated as a time step with 28 features, allowing the network to learn how visual patterns evolve across the vertical dimension of the image. This allows the model to capture relationships between different regions of the hand, which often span multiple rows of pixels. Using LSTM is a way to explore different methods for analyzing image data. While convolutional neural networks are better suited for image classification tasks, the LSTM provides a baseline for evaluating how well sequential modeling can capture the structure of sign language gestures.

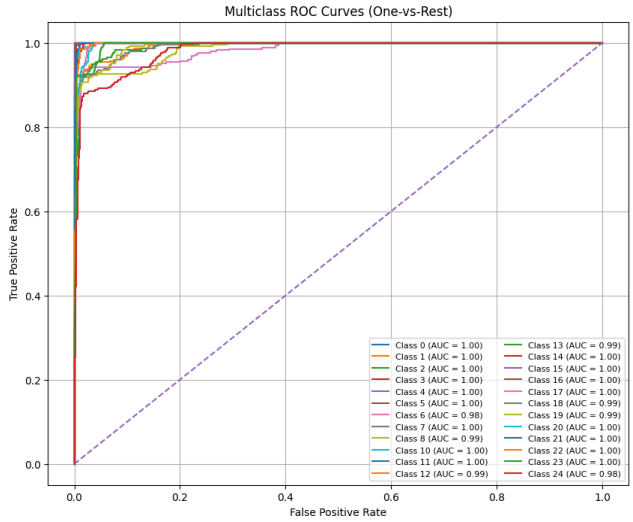
The LSTM model demonstrated rapid learning across all folds, with training accuracy increasing sharply within the first few epochs. In the cross-validation results, validation accuracy exceeded 0.98 after eight to ten epochs and reached values as high as 0.9994 in some folds. The mean holdout accuracy across all five folds was 0.9916, indicating that the model performs extremely well on subsets of the training data that were not used during fitting. This suggests that the model is capable of capturing strong patterns within the dataset and generalizing in the training data.

The training and validation accuracy curves support this because both curves increase smoothly and remain closely aligned throughout training. The absence of a large gap during cross-validation indicates that the rotational augmentation was effective in minimizing overfitting. The smooth convergence of both curves suggests that the optimization process was stable and that the model was appropriate.





Despite the strong cross-validation performance, the final model achieved a test accuracy of 0.8935 on the independent test set. This discrepancy between cross-validation accuracy and test accuracy suggests that the model does not generalize as effectively to the true test distribution as it does within the training folds. While cross-validation results showed near-perfect performance, the test results reveal that the model struggles with certain distinctions when evaluated on unseen data.



The one-vs-rest ROC analysis provides additional insight into the model's performance. The ROC curves for most classes are very close to the top-left corner of the plot, and the macro-average AUC of 0.9958 indicates that the model has discriminative ability overall. This means that, in terms of ranking predictions, the model is highly effective at separating classes even with classification errors.

However, the high AUC values alongside lower classification accuracy highlight an important distinction. While the model assigns high probabilities to the correct classes in most cases, the final predicted label may still be incorrect when competing classes have similar probability values. The primary challenge is distinguishing between visually similar gestures rather than the general structure of the hand. A key limitation of the LSTM approach is that it processes images in a sequential manner, which could lead to a loss of spatial information. Unlike convolutional neural networks, which explicitly model local spatial relationships through filters, the LSTM relies on sequential dependencies that may not fully capture two-dimensional structure. This limitation becomes particularly apparent when distinguishing between gestures that differ only in small, localized features.

Conclusion

The results of our models show the impact of model architecture and training strategies on ASL image classification. The MLP served as a baseline by performing well on the training and validation data after tuning the learning rate and model depth. However, its lower test accuracy (~0.5417) signifies MLPs' inability to fully capture spatial relationships in images and distinguish subtle differences between gestures.

CNN models, on the other hand, consistently outperformed the MLP. Across the multiple CNN architecture variations, there were many trade-offs between model complexity, parameter efficiency, and generalization, however, wider networks achieved the highest validation (0.9995) and test (0.9787) accuracy. Other models like the residual connections and global pooling model were parameter efficient and had an equally strong performance, but not as high as the wider network. These results indicate that CNNs are well-suited to capture the fine-grained differences in hand shapes necessary for accurate ASL classification.

Even though the LSTM model is designed for sequential data, it provided important insights into alternative approaches. The model was able to achieve a strong cross-validation accuracy (mean holdout 0.9916), but lower test accuracy (0.8935). This suggests that processing images sequentially may fail to fully preserve two-dimensional spatial relationships. However, the high ROC-AUC (0.9958) shows that the model assigns probabilities effectively, but misclassifications occur when classes have visually similar features.

Data augmentation further improved generalization, particularly for CNN models. Mild transformations allowed the network to learn robust representations without distorting the subtle hand gestures critical to classification, allowing the model to generalize better to unseen data.

Overall, these findings for all 3 types of models suggest that convolutional neural networks (CNNs), with wider architectures (Wide CNN) are the most effective choice for ASL image classification, balancing accuracy, generalization, and the ability to capture complex spatial features.